

演算法報告

楊文德

● Heap sort

```
// 定義 Array 結構，用來存堆排序的資料
struct Array {
    int heap_size = 0; // 堆的大小
    vector<int>& v;      // 引用外部的 vector，避免 address 導致更改一個全都變化的現象
    Array(vector<int>& vec) : v(vec) {} // 建構子，綁定引用
};
```

我先定義一個 structure，用來存儲 Heap 的大小以及陣列。

```
// 建立最大堆
void Build_max_heap(Array& a) {
    a.heap_size = a.v.size(); // 設定堆大小
    for (int i = (a.v.size() - 1) / 2; i >= 0; i--) {
        Max_heapify(a, i); // 從最後一個非葉節點開始調整
    }
}
```

另陣列大小前半作為 tree 的根，然後再對每一個部分做 Heap 化。

```
// 調整函數，維護最大堆性質
void Max_heapify(Array& a, int i) {
    int l = 2 * i + 1; // 左子節點
    int r = 2 * i + 2; // 右子節點
    int largest = i;   // 假設當前節點最大
    if (l < a.heap_size && a.v[l] > a.v[i]) {
        largest = l; // 更新最大值為左子節點
    }
    if (r < a.heap_size && a.v[r] > a.v[largest]) {
        largest = r; // 更新最大值為右子節點
    }
    if (largest != i) {
        swap(a.v[i], a.v[largest]); // 交換，使父節點最大
        Max_heapify(a, largest);    // 遞迴調整受影響的子樹
    }
}
```

對一個最小單元的 Heap tree 做驗證，使其的兩個 leafs 都能遵守 parents ≥ 2 個 children 的關係。

```
// 執行堆排序並測量時間
void Heapsort(Array& a) {
    double time = 0;
    LARGE_INTEGER nFreq, nBeginTime, nEndTime;
    QueryPerformanceFrequency(&nFreq); // 取得計時器頻率
    QueryPerformanceCounter(&nBeginTime); // 開始計時
    Build_max_heap(a); // 建立最大堆
    for (int i = a.v.size() - 1; i >= 1; i--) {
        swap(a.v[0], a.v[i]); // 將最大值放到陣列末尾
        a.heap_size--;        // 減少堆大小
        Max_heapify(a, 0);    // 調整堆
    }
    QueryPerformanceCounter(&nEndTime); // 停止計時
    time = (double)(nEndTime.QuadPart - nBeginTime.QuadPart) / (double)nFreq.QuadPart;
    cout << fixed << setprecision(6) << "Heap Sort Time : " << time << " seconds" << endl;
}
```

建立完 Heap tree 之後我們再讓其還原成排序過後的矩陣。

● 計時方法

Windows 系統計時方法

透過 Windows 系統的高精度計時器 QueryPerformanceCounter 實現計時功能，精確度可達到微秒等級。

```
#include <stdio.h>
#include<windows.h>
#include <unistd.h>

int main() {
    double time = 0;
    LARGE_INTEGER nFreq, nBeginTime, nEndTime;
    QueryPerformanceFrequency(&nFreq);
    QueryPerformanceCounter(&nBeginTime); // 開始計時

    sleep(3);

    QueryPerformanceCounter(&nEndTime); // 停止計時
    time = (double)(nEndTime.QuadPart - nBeginTime.QuadPart) / (double)nFreq.QuadPart; // 計算程
    printf("Elapsed time in seconds: %lf s\n", time);
}
```

這方法主要是透過 window 系統去計算 CPU 運算的次數，然後再透過這 BeginTime 以及 EndTime 之間的次數差除以頻率即是這段程式運行了多少時間，精確度可以達到微秒等級。

● 樣本數與時間比較

```
Please enter the size of an array : 100
Heap Sort Time : 0.000017 seconds
Insertion Sort Time : 0.000011 seconds
Merge Sort Time : 0.000085 seconds
Please enter the size of an array : 1000
Heap Sort Time : 0.000191 seconds
Insertion Sort Time : 0.000873 seconds
Merge Sort Time : 0.000837 seconds
Please enter the size of an array : 10000
Heap Sort Time : 0.003367 seconds
Insertion Sort Time : 0.109401 seconds
Merge Sort Time : 0.035837 seconds
Please enter the size of an array : 100000
Heap Sort Time : 0.034277 seconds
Insertion Sort Time : 10.902137 seconds
Merge Sort Time : 0.097630 seconds
Please enter the size of an array : 1000000
Heap Sort Time : 0.489701 seconds
Insertion Sort Time : 1176.357292 seconds
Merge Sort Time : 3.331697 seconds
```

● 時間複雜度分析

1. 從 100 增加到 1000 (10 倍)，時間從 0.000017 增加到 0.000191，約 11.2 倍。
2. 從 1000 到 10000 (10 倍)，時間從 0.000191 到 0.003367，約 17.6 倍。
3. 從 10000 到 100000 (10 倍)，時間從 0.003367 到 0.034277，約 10.2 倍。
4. 從 100000 到 1000000 (10 倍)，時間從 0.034277 到 0.489781，約 14.3 倍。

從上述來說，我們首先可以排除掉 $O(n^2)$ 複雜度的演算法，因為理論上 $O(n^2)$ 的演算法的樣本數以 10 倍速度增長的話，所需時間也會相對的以 100 倍速度增長，接下來我們可以從成長倍速發現與 merge sort 相似

1. 從 100 到 1000 (10 倍)，時間從 0.000085 到 0.000873，約 10.3 倍。
2. 從 1000 到 10000 (10 倍)，時間從 0.000873 到 0.053837，約 61.6 倍。
3. 從 10000 到 100000 (10 倍)，時間從 0.053837 到 0.976330，約 18.1 倍。
4. 從 100000 到 1000000 (10 倍)，時間從 0.976330 到 3.331697，約 3.4

倍。

除去 61. 以及 3.4 倍事件的異常(可能是記憶體配置的影響)，可以發現結果與 heap sort 的倍率差不多，因此我們可以大膽猜測結果為 $O(n \log(n))$ 。

- 心得

在這次的 HW 中我終於迎來第一次用 tree 的概念來排序的方法，將講義上的 pseudocode 轉換成能夠實踐的程式碼真的很有成就感。

- 引用

時間計算

<https://andy6804tw.github.io/2022/09/23/measure-elapsed-time/>